Ex. No.: 6d)
Date 2 |03|2025

ROUND ROBIN SCHEDULING

Aim:

To implement the Round Robin (RR) scheduling technique

Algorithm:

1. Declare the structure and its elements.
2. Get number of processes and Time quantum as input from the user.
3. Read the process name, arrival time and burst time
4. Create an array **rem_bt[]** to keep track of remaining burst time of processes which is initially copy of bt[] (burst times array)
5. Create another array **wt[]** to store waiting times of processes. Initialize this array as 0. 6. Initialize time : t = 0
7. Keep traversing the all processes while all processes are not done. Do following for i'th process if it is not done yet.
   a- If rem_bt[i] > quantum
   (i) t = t + quantum
   (ii) bt_rem[i] -= quantum;
   b- Else // Last cycle for this process
   (i) t = t + bt_rem[i];
   (ii) wt[i] = t - bt[i]
   (iii) bt_rem[i] = 0; // This process is over
8. Calculate the waiting time and turnaround time for each process.
9. Calculate the average waiting time and average turnaround time.
10. Display the results.

Program Code:

```
#include <stdio.h>
int main() {
    int n, quantum;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int processes[n], bt[n], at[n], wt[n], tat[n],
                                    rem_bt[n];

    printf("Enter Time Quantum: ");
    scanf("%d", &quantum);
```

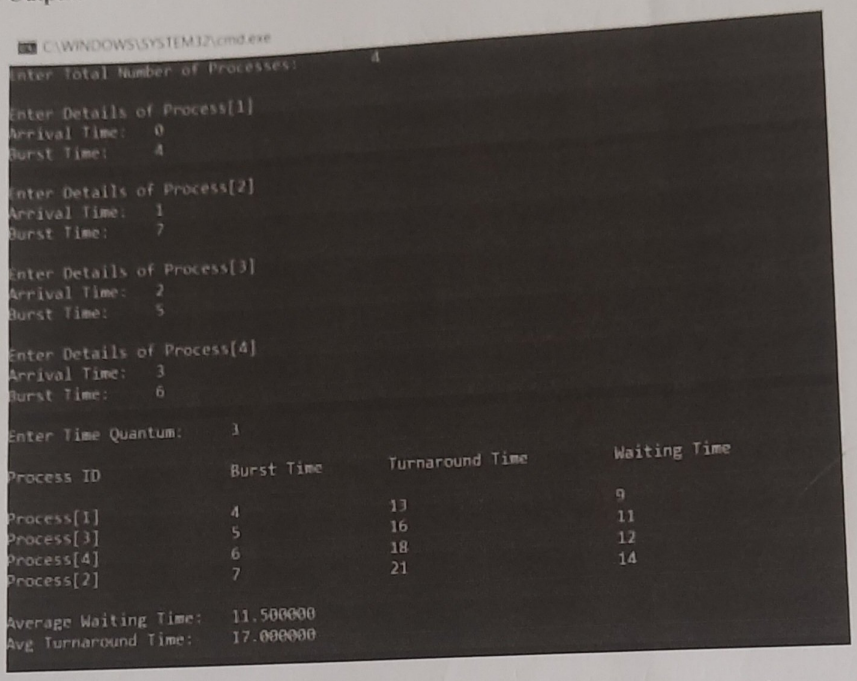
```
for (int i=0; i<n; i++) {
    Processes[i] = i+1;
    printf ("Enter Arrival Time and Burst Time for
                                    Process %d: ", i+1);
    scanf ("%d%d", &at[i], &bt[i]);
    rem_bt[i] = bt[i];
    wt[i] = 0;
}
int t = 0;
int done;
do {
   done = 1;
  for (int i=0; i<n; i++) {
      if (rem_bt[i] > 0) {
          done = 0;
        if (rem_bt[i] > quantum) {
            t += quantum;
            rem_bt[i] -= quantum;
        } else {
            t += rem_bt[i];
            wt[i] = t - bt[i] - at[i];
            rem_bt[i] = 0;    }
      }
   }
}
```

```
while (!done);
float total_wt = 0, total_tat = 0;
printf ("\nProcess \t AT\t BT \t WT\t TAT\n");
for (int i = 0; i<n; i++) {
    tat[i] = bt[i] + wt[i];
    total_wt += wt[i];
    total_tat += tat[i];
    printf ("p%d\t%d\t%d\t %d\t %d\t%d\n",
                processes[i], at[i], bt[i], wt[i]
                                    tat[i]);

}
printf ("\n Average Waiting Time : %6.2f", total_wt/n
printf ("\n Average TurnAround time : %6.2f\n",
                                    total_tat /n);

return 0;
}
```

Sample Output:



INPUT!

Enter number of processes: 4

Enter Time Quantum : 3

Enter Arrival Time and Burst Time for Process P1 : 0 5

Enter Arrival Time and Burst Time for Process P2 : 1 3

Enter Arrival Time and Burst Time for Process P3 : 2 8

Enter Arrival Time and Burst Time for Process P4 : 3 6

Process	AT	BT	WT	TAT
P1	0	5	9	14
P2	1	3	2	5
P3	2	8	12	20
P4	3	6	11	17

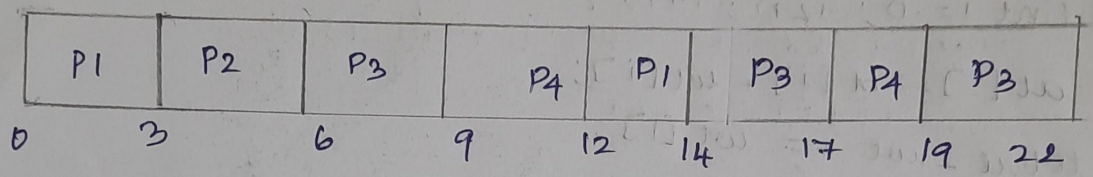
Average Waiting Time : 8.50 ms

Average TurnAround Time : 14.00 ms

Result:

Thus the code to implement the Round Robin technique has been executed successfully

Gantt chart:



Tabulation:

Process	AT (ms)	BT (ms)	CT (ms)	WT = TAT - BT (ms)	TAT = CT - AT (ms)
P1	0	5	14	6	11
P2	1	3	6	4	7
P3	2	8	22	9	17
P4	3	6	19	9	15

Average WT = 8.50 ms

Average TAT = 14.00 ms